数字电路笔记

预备知识 ⟶ 组合逻辑电路 ⟶ 时序逻辑电路

不连续、是固定量整数倍的量称为数字量，与数字量对应的信号称为数字信号，处理数字信号的电路称为数字电路。

第一章　预备知识

一、数制

1. 不同的数制

数制（符号）	基数	数码
二进制 B	2	0,1
八进制 O	8	0-7
十进制 D	10	0-9
十六进制 H	16	0-9, A-F

2. 数制转换

非十进制互转

每 3 位或 4 位分组，然后展开或缩写即可。例：

$$(11001101.011)_2 = (315.3)_8 = (CD.6)_{16}$$

非十进制转十进制

按权展开即可。例：

$$(101.01)_2 = 1\times 2^2 + 0\times 2^1 + 1\times 2^0 + 0\times 2^{-1} + 1\times 2^{-2} = (5.25)_{10}$$

$$(73.4)_8 = 7\times 8^1 + 3\times 8^0 + 4\times 8^0 + 4\times 8^{-1} = (59.5)_{10}$$

$$(3\mathrm{D.C})_{16} = 3\times 16^1 + 13\times 16^0 + 12\times 16^{-1} = (61.75)_{10}$$

十进制转非十进制

整数部分除基取余，从低到高；小数部分乘基取整，从高到低。

例：13.25

13 除以 2 余 1，6 除以 2 余 0，3 除以 2 余 1，1 除以 2 余 1，整数部分为 1101；

0.25 乘以 2 得 0.5，取整为 0，0.5 乘以 2 得 1.0，取整为 1，小数部分为 01；

故 $(13.25)_{10} = (1101.01)_2$

二、编码

1. 原码、反码与补码

编码类型	描述	缺点
原码	首位表示符号位，剩余位为二进制绝对值	减法运算困难
反码	正数与零不变；负数的符号位不变，其余位为原码取反	有两个反码表示零
补码	正数与零不变；负数的补码为反码加 1	无

补码取负

对补码取负数的方法：对补码再取一次补码。

2. 二进制编码 **二进制数码不一定表示二进制数**。

最常见的二进制编码是自然二进制码，即将其他数制的数转为二进制后的结果，不赘述。

3. 二-十进制编码（BCD 码）

编码类型	权	特性
8421 码	8, 4, 2, 1	简便直观
2421 码	2, 4, 2, 1	对 9 自补码
余 3 码	无（由 8421 码 +3 得到）	对 9 自补码
5421 码	5, 4, 2, 1	可以产生对称方波输出
Gray 码	无	相邻两数码只有一位二进制数不同

二-十六进制自然编码与 8421BCD 码区别

二-十六进制自然编码是将十进制数直接转换为十六进制数，而 8421BCD 码是将十进制数的每一位分别转换为 4 位二进制数。例：$(53)_{10} = (101011)_2 = (01010011)_{8421\text{BCD}}$

4. 可靠性编码

(1) Gray 码

Gray 码是一种特殊的二进制编码，其特点是相邻两个数码之间只有一位二进制数不同。Gray 码的主要优点是减少了在数字电路中由于信号变化引起的误差。

4 位反射 Gray 码的编码表如下：

0 (0000)	1 (0001)	2 (0011)	3 (0010)
4 (0110)	5 (0111)	6 (0101)	7 (0100)
8 (1100)	9 (1101)	10 (1111)	11 (1110)
12 (1010)	13 (1011)	14 (1001)	15 (1000)

将 9 的 Gray 码换为 1000，即得到 Gray BCD 码。

(2) 奇偶校验码

在要传输的数据中增加一个校验位，使得数据中 1 的个数为奇数或偶数，从而实现对数据传输错误的检测。

最常用的是奇校验码，因其使得全 0 成为非法码，便于检测。

由于异或门奇 1 偶 0 的特性，往往用异或门实现奇偶校验码的生成。

(3) 其他可靠性编码：Hamming 码等

三、逻辑代数

逻辑常量：0, 1

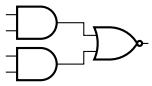
基本逻辑运算：与（AND）、或（OR）、非（NOT）

（一）逻辑运算

1. 基本逻辑运算

逻辑门	运算规则	逻辑运算式	IEEE 图形符号	IEC 图形符号
与门（AND）	有 0 则 0	$F(A, B) = A \cdot B$		
或门（OR）	有 1 则 1	$F(A, B) = A + B$		
非门（NOT）	反转输入	$F(A) = \overline{A}$		

2. 复合逻辑运算

逻辑门	运算规则	逻辑运算式	IEEE 图形符号	IEC 图形符号
与非门（NAND）	有 0 则 1	$F(A, B) = \overline{A \cdot B}$		
或非门（NOR）	有 1 则 0	$F(A, B) = \overline{A + B}$		
与或非门（AOI）	与、或、非	$F(A, B, C, D) = \overline{AB + CD}$		
异或门（XOR）	同 0 异 1	$F(A, B) = A \oplus B = \overline{A} \cdot B + A \cdot \overline{B}$		
同或门（XNOR）	同 1 异 0	$F(A, B) = A \odot B = \overline{A} \cdot \overline{B} + A \cdot B$		

（二）逻辑代数公式

1. 基本公式

名称	或式	与式
交换律	$A+B=B+A$	$A\cdot B=B\cdot A$
结合律	$(A+B)+C=A+(B+C)$	$(A\cdot B)\cdot C=A\cdot(B\cdot C)$
分配律	$A+(B\cdot C)=(A+B)\cdot(A+C)$	$A\cdot(B+C)=(A\cdot B)+(A\cdot C)$
吸收律	$A+(A\cdot B)=A$	$A\cdot(A+B)=A$
0-1 律	$A+1=1,\quad A+0=A$	$A\cdot 0=0,\quad A\cdot 1=A$
互补律	$A+\overline{A}=1$	$A\cdot\overline{A}=0$
重叠律	$A+A=A$	$A\cdot A=A$
还原律	$A+\overline{A}=1$	$A\cdot\overline{A}=0$
反演律	$\overline{A+B}=\overline{A}\cdot\overline{B}$	$\overline{A\cdot B}=\overline{A}+\overline{B}$

2. 常用公式

   - $A\cdot B+A\cdot\overline{B}=A$
   - $A+\overline{A}\cdot B=A+B$
   - $A\cdot B+\overline{A}\cdot C+B\cdot C=A\cdot B+\overline{A}\cdot C$
   - $(A+B)\cdot\left(\overline{A}+C\right)=A\cdot C+\overline{A}\cdot B$
   - $\overline{\overline{A}\cdot B+A\cdot\overline{B}}=\overline{A}\cdot\overline{B}+A\cdot B$，即 $\overline{A\oplus B}=A\odot B$

（三）逻辑代数基本规则

1. 代入规则：用逻辑表达式 F 代入某个变量 X，新逻辑表达式仍然成立。
2. 反演规则

   将原函数 F 作如下代换：

   - $+\leftrightarrow\cdot$
   - $1\leftrightarrow 0$
   - $A\leftrightarrow\overline{A}$

   即得到反函数 $\overline{F}$。反函数输出的值**与原函数相反**。
3. 对偶规则

   将原函数 F 作如下代换：

   - $+\leftrightarrow\cdot$
   - $1\leftrightarrow 0$

   即得到对偶函数 F'。对偶函数**不需要代换反变量**。

   若 F 与 G 相等，则 F' 与 G' 也相等。

（四）正负逻辑

- 正逻辑：逻辑 1 对应高电平，逻辑 0 对应低电平。
- 负逻辑：逻辑 1 对应低电平，逻辑 0 对应高电平。

正负逻辑转换

对自变量预先取反，即能将负逻辑电路转换为正逻辑电路，反之亦然。

（五）逻辑函数

1. 表示方法
   - 真值表
   - 逻辑表达式
   - 逻辑电路图
   - Karnaugh 图
   - 波形图
   - 硬件描述语言
2. 标准表达式
   - 最简与或式

     性质：所有最小项中只有一项可能是 1；$m_i \cdot m_j = 0, \quad i \neq j$；$\sum_{i=0}^{2^n-1} m_i = 1$
   - 最简或与式

     性质：所有最大项中只有一项可能是 0；$M_i + M_j = 1, \quad i \neq j$；$\prod_{i=0}^{2^n-1} M_i = 0$

（六）逻辑函数的化简

1. 代数法根据逻辑代数公式，对逻辑函数进行化简。常见方法有：
   - 并项法：$AB + A\overline{B} = A$
   - 吸收法：$A + AB = A$、$AB + \overline{A}C + BC = AB + \overline{A}C$

     还可推广为

     $$AB + \overline{A}C + BC[f(D, E, \cdots)] = AB + \overline{A}C$$
   - 消元法：$A + \overline{A}B = A + B$
2. Karnaugh 图法

Karnaugh 图的数学结构

(a) **输入空间**：对于 n 个逻辑变量，所有输入组合构成二进制空间

$$\{0,1\}^n,$$

可视为 n 维超立方体 Q_n 的 2^n 个顶点。两顶点 x, y 的 Hamming 距离为

$$d_H(x,y) = \sum_{i=1}^{n} |x_i - y_i|.$$

当 $d_H(x,y) = 1$ 时，两输入组合仅有一个变量不同，对应超立方体中的一条边。

(b) Gray **坐标**：Karnaugh 图按 Gray 码排列各坐标，使相邻码字满足

$$d_H = 1.$$

因而 Karnaugh 图中的几何邻接对应逻辑空间中的边邻接，即

$$\boxed{\text{几何相邻} \iff \text{仅一个逻辑变量发生变化}}.$$

图的上下、左右边界同样相邻，因此 Karnaugh 图具有周期边界。

(c) Descartes **积分解**：将 n 个变量分为两组，分别作为行、列坐标，则

$$Q_n \cong Q_p \square Q_q, \qquad p + q = n,$$

其中 $\square$ 表示图的笛卡尔积。例如四变量 Karnaugh 图满足

$$Q_4 \cong Q_2 \square Q_2,$$

行、列均按

$$00,\ 01,\ 11,\ 10$$

的 Gray 顺序排列，从而将四维二进制空间表示为二维坐标表。

(d) **子立方体与矩形**：一个逻辑乘积项中，未出现的变量可以自由取 0 或 1。若有 k 个自由变量，则该项对应一个含

$$2^k$$

个顶点的 k 维子立方体。

若其中 r 个自由变量属于行坐标，其余 $k-r$ 个属于列坐标，则该子立方体在 Karnaugh 图中表现为

$$2^r \times 2^{k-r}$$

的矩形，因此其面积恒为

$$\boxed{2^k}.$$

(e) **代数消元**：相邻顶点仅在一个变量上互补，因此可利用

$$AX + A\overline{X} = A(X + \overline{X}) = A$$

消去该变量。每使一个变量自由变化，相当于沿一个坐标方向扩展一维，因此

$$\boxed{2^k \text{ 个相邻格组成的子立方体} \iff \text{消去} k \text{ 个变量}}.$$

例如

$$AB\overline{C} + ABC = AB(\overline{C} + C) = AB,$$

几何上对应两个顶点沿 C 方向合并为一条边。

(f) **与或式的几何意义**：一个乘积项对应 $\{0,1\}^n$ 中一个轴对齐的子立方体；逻辑或对应这些子立方体的集合并。因此

$$F = P_1 + P_2 + \cdots + P_m$$

可理解为用若干子立方体覆盖

$$F^{-1}(1) = \{x \in \{0,1\}^n \mid F(x) = 1\}.$$

Karnaugh 图化简的本质即为：

$$\boxed{\text{用尽可能大的子立方体覆盖全部1 格}}.$$

对偶地，或与式则通过圈取 0 格，用子立方体描述 $F^{-1}(0)$。

逻辑代数	离散几何
n 个变量	n 维超立方体Q_n
一个输入组合	一个顶点
$d_H = 1$	一条边
最小项	一个顶点
一个变量自由	沿一维扩展
2^k 项合并	k 维子立方体
消去k 个变量	增加k 个自由维度
逻辑或	集合并
与或式化简	用大子立方体覆盖$F^{-1}(1)$

$$\boxed{\text{Karnaugh 图} = \text{以 Gray 码保持邻接关系的超立方体二维坐标化}}$$

第二章　门电路

特殊名词：

- 灌电流：流入输出端的电流。
- 拉电流：流出输出端的电流。
- 扇出系数：门电路能驱动同类门的个数。

一、二极管电路

二、CMOS 门

MOS 管的数字电路特性

MOS 管类型	栅极（G）	源极（S）	漏极（D）
NMOS	输入	低电平（GND）	输出
PMOS	输入	高电平（V_{DD}）	输出

NMOS 低通高断，PMOS 高通低断。

输入电平	NMOS	PMOS	输出电平
低电平（0）	截止	导通	高电平（1）
高电平（1）	导通	截止	低电平（0）

（一）CMOS 反相器（非门）

1. 电路结构：PMOS+NMOS，栅极（G）作为输入，漏极（D）作为输出。
2. 特性：**静态功耗极小**，只有在输入电平变化时才有功耗。
3. 输入为下拉电阻时为低电平，输入为上拉电阻时为高电平，不可悬空。

（二）CMOS 与非门和或非门

与非门	或非门
两 PMOS 并联，两 NMOS 串联	两 PMOS 串联，两 NMOS 并联

（三）CMOS 与或非门

由两个与非门线与得到。

（四）CMOS 漏极开路门（OD 门）

1. 电路结构：必须有上拉电阻，才能输出高电平。
2. 特性：能够实现线与。

如何计算上拉电阻范围

两个边界条件：

1. 上一级门电路均输出高电平；
2. 上一级门电路只有一路输出低电平。

故有如下公式

$$\frac{V'_{\mathrm{DD}}-V_{\mathrm{OL\,max}}}{I_{\mathrm{OL\,max}}-mI_{\mathrm{IL}}}\le R_{\mathrm{L}}\le\frac{V'_{\mathrm{DD}}-V_{\mathrm{OH\,min}}}{nI_{\mathrm{OH}}+mI_{\mathrm{IH}}}$$

（五）CMOS 模拟开关

1 通 0 断。

三、TTL 门

晶体管的数字电路特性

（一）TTL 与非门

特性：输入悬空为 1；上拉电阻可以为 0 或 10kΩ，下拉电阻 $R_{\mathrm{I}}\le 300\Omega$。

饱和深度 $S=\dfrac{I_B}{I_BS}$ 饱和越深，导通越充分，压降越小；但存储电荷越多，关断越慢。

A	B	T_1	T_2	T_3	T_4	F
L	L	饱和	截止	放大	截止	H
L	H	饱和	截止	放大	截止	H
H	L	饱和	截止	放大	截止	H
H	H	倒置	饱和	截止	饱和	L

【技巧】快速判断 TTL 门电路的逻辑功能

对于晶体管来说，从基极（B）输入的信号，从集电极（C）输出的信号反相，从发射极（E）输出的信号同相。

TTL 或非门、与或非门和异或门都能用这种技巧判断逻辑功能。

四、ECL 门

ECL 门的特点是**速度快**，但功耗大，输出端可以实现**线或**功能。

门内部共射实现线或，共集实现线与。

三种门的性质比较

- 平均传输延迟时间 t_{pd}：$t_{pd}(\text{ECL}) < t_{pd}(\text{TTL}) < t_{pd}(\text{CMOS})$

  工作速度取反

- 噪声容限：ECL<MOS<TTL

  抗干扰能力相同。

- 静态能耗

  MOS<TTL<ECL

五、三态门

三态门除输出高电平 1、低电平 0 外，还能输出高阻态 z。

特性：电平类型与使能端正反一致，即：EN 对应高电平有效，$\overline{EN}$ 对应低电平有效，否则为高阻态 z。

应用：

- 分时复用：将两个使能端类型相反的三态门输出端相连，从而用使能信号控制输出端的信号类型。
- 双向传输：将两个使能端类型相反的三态门输出端相连，从而用使能信号控制输出端信号的传输方向。

第三章　组合逻辑电路

一、小规模组合电路（Small Scale Integrated Circuites, SSI）

小规模组合电路是指集成电路中逻辑门的数量不超过 10 个的组合逻辑电路。

小规模组合电路的表达式可以整理为与非 · 与非式或或非 · 或或式，从而实现为双轨输入电路。

1. 双轨输入电路

- 与非 · 与非式

  可以运用反演律将最简与或式化为与非 · 与非式，用两级与非门实现。

  例：$F = A\overline{B} + AC = \overline{(\overline{A\overline{B}}) \cdot (\overline{AC})}$

- 或非 · 或非式

  可以运用反演律将最简或与式化为或非 · 或非式，用两级或非门实现。

  例：$F = (A + \overline{B})(A + C) = \overline{(\overline{A + \overline{B}}) + (\overline{A + C})}$

与非 · 与非式和或非 · 或非式各有优势，不存在绝对的优劣。判断哪种方式能最大程度简化时，应根据具体逻辑函数设计比较。

2. 单轨输入电路

单轨输入电路可以通过给每个输入加非门实现为双轨输入电路，不多叙述。

二、冒险

（一）逻辑冒险

逻辑冒险的判断与消除方法

1. 代数式法

只有同时存在正反变量（如 A 与 $\overline{A}$）的逻辑函数才可能出现逻辑冒险。

将标准与或式/或与式中该正反变量以外的所有变量设为 0 或 1，将逻辑函数化为

$$F = A + \overline{A}$$

的形式，此时存在偏 0 型冒险；或化为

$$F = A \cdot \overline{A}$$

的形式，此时存在偏 1 型冒险。

若要消除逻辑冒险，则在标准与或式/或与式中增加冗余项即可。荣誉项确定办法为将存在逻辑冒险时的所有变量组合为与式或或式，作为冗余项加入原逻辑函数中。

例：$B = 0, C = 1$ 时存在偏 0 型冒险，冗余项为 $+\overline{B}C$；$B = 0, C = 1$ 时存在偏 1 型冒险，冗余项为 $\cdot(\overline{B} + C)$。

2. Karnaugh 图法

判断 Karnaugh 图中是否存在相切圈，若存在则说明存在逻辑冒险现象。

画圈使相切圈相接，即可消除逻辑冒险现象。

（二）功能冒险

在组合电路中，当有**两个或两个以上输入变量同时**发生变化，变化前后电路的输出相同，而在输入变量发生变化时可能出现瞬时错误输出，这种现象称为静态功能冒险

由状态 A 到状态 B 的过程中，若首尾电平一致，中间出现不一样的电平，则称为功能冒险。功能冒险的决定因素如下：

(1) 变量变化两个及以上；

(2) 首尾电平状态一致（从 0 到 0，从 1 到 1）；

(3) 中间电平变化（从 0 到 1，从 1 到 0）。

消除功能冒险的方法包括：

- 添加滤波电容，但会破坏输出波形；
- 加选通脉冲，对**逻辑冒险和功能冒险都有效**。

> **注意**：添加冗余项只能消除逻辑冒险而非功能冒险！

逻辑冒险与功能冒险合称静态冒险。

（三）动态冒险

在输入变化前、后函数值不同；且在输入信号变化瞬间，输出将变化三次或更高的奇数次。此处不讨论。

三、多输出函数

1. 用卡诺图分别对每个函数进行化简，并用箭头连线表示出所有的公用圈；
2. 从各个函数**相同最小项**出发，试图改变原来圈法，以求得更多的公用圈。

   原则：

   - 若改圈法后能使总圈数减少（指不同圈），则改；
   - 若使总圈数增加，则不改；
   - 若总圈数不变，则取最大的合并圈，使变量输入端减少；
   - 这一工作可重复进行多次。**注意**：**单个变量的圈不用修改**

3. 写出多输出函数的表达式，并绘出逻辑图。

四、中规模组合电路（Medium Scale Integrated Circuites, MSI）

（一）数码比较器

1. TTL 中规模集成 4 位数码比较器 74LS85

   直观理解即可，比较两个 4 位二进制数的大小关系，输出 $A>B$、$A=B$、$A<B$；额外设 G'（大于）、E'（等于）、S'（小于）输入端，表示两数相等时的输出。

   Verilog HDL 实现代码如下：

```
module compare_4(Y, A, B);
    input [3:0] A, B;
    output [2:0] Y;
    reg [2:0] Y;

    always @(A or B)
    begin
        if (A > B)
```

```
            Y = 3'b100;
        else if (A == B)
            Y = 3'b010;
        else
            Y = 3'b001;
    end
endmodule

```

功能扩展：

- 5 位比较器

  使用一片元件，将两个 5 位二进制数的高 4 位输入到比较器中，低 1 位输入到比较器的 $A > B$、$A < B$ 端口中；不可以比较相等数值。

- 串行级联比较器

  将多片比较器的输出端口与 G'、E'、S' 端口连接起来，即可实现串行级联比较器。

  注意：前后顺序为**先低后高**，即第一级电路比较最低 4 位；第一级电路应将 G'、S' 接地而 E' 接高电平，保证第一级电路正确输出相等结果。

- 并行级联比较器

  基于 5 位比较器修改得来。使用 5 片 4 位比较器并联作为一级电路，1 片 4 位比较器作为二级电路。

  除最低 4 位接 $E' = 1$ 保证相等输出外，一级电路比较器输入方法如 5 位比较器，最多可以比较 24 位二进制数；二级电路输入法也同 5 位比较器，一级电路的 $A > B$ 端口接到二级电路的 a_i 端口，$A < B$ 端口接到二级电路的 b_i 端口，最低四位输出接到 G'、E'、S' 端口。

（二）编码器与优先编码器

1. 二-十位编码器 C304

   真值表即 8421BCD 码。

$$D = 8 + 9$$

$$C = 4 + 5 + 6 + 7$$

$$B = 2 + 3 + 6 + 7$$

$$A = 1 + 3 + 5 + 7 + 9$$

2. 8 线-3 线编码器（略）

3. 优先编码器 74148

   功能描述：

   输入端口 $I_0 \sim I_7$，使能端 $\overline{ST}$；输出端口 $Y_0 \sim Y_2$，使能输出端 Y_S，扩展输出端 $\overline{Y_{EX}}$。

   若使能端为 1，全部输出 1，代表停止输出；

   若使能端为 0，输入端全为 1，输出端除 Y_S 是 0 外，其余均为 1，代表没有输出信号；

若使能端为 0，输入端有信号，则输出端 Y_S 为 1，输出端 $Y_0 \sim Y_2$ 为输入端中最高位的二进制编码。

简记为：非编码状态时 $\overline{Y_{EX}}$ 为 1；编码状态且无输入信号时 Y_S 为 0；其他状态编码最高位。

功能扩展：

- 16 线至 4 线的优先编码器

  2 片 74148 串联，第一片编码 8 15，第二片编码 0 7；第一片的输出端接到第二片的使能端。

  输出端为**反码**$F_0 \sim F_3$，扩展端 F_{EX}。

  片 1 的 $\overline{Y_{EX}}$ 作为 4 位码最高位 F_3 输出，两片对应位 $Y_2 \sim Y_0$ 相与作为低 3 位 $F_2 \sim F_0$ 输出，两片扩展段 $\overline{Y_{EX}}$ 相与作为低 3 位 F_{EX} 输出，

- 8421BCD 码优先编码器

  10 输入 4 输出。

  $I_0 \sim I_7$ 正常通过非门后从 $F_0 \sim F_2$ 输出，I_8、I_9 与非后从 F_3 输出，I_9 和 Y_0 与非后从 F_0 输出。

（三）译码器

1. 2 线至 4 线译码器

```
module decode24(a, led);
  input [1:0] a;
  output [3:0] led;
  reg [3:0] led;

  always @(a)
  begin
      case (a)
          2'b00: led = 4'b0001;
          2'b01: led = 4'b0010;
          2'b10: led = 4'b0100;
          2'b11: led = 4'b1000;
      endcase
  end
endmodule

```

2. 三线至八线译码器 74LS138

   S_A 为 1，S_B、S_C 为 0 时，能正常译码，否则输出端全为 1，表示无输出。

```
module decoder3to8(out, in);

    output [7:0] out;
    input [2:0] in;

    assign out = 1'b1 << in;  // 根据输入进行左移，输出'1'有效
endmodule

```

```
module decoder3to8(sw, led);
    input [2:0] sw;             // 开关输入信号，作为3-8译码器的输入信号控制LED
    output [7:0] led;
    reg [7:0] led;              // 在always过程中只能对reg型变量赋值

    always @(sw)                // always过程块，括号中sw为敏感变量；
                                //当sw变化一次，执行一次always中所有语句；否则保持不变
    begin
        case(sw)                // case语句
            3'b000: led = 8'b0111_1111;
            3'b001: led = 8'b1011_1111;
            3'b010: led = 8'b1101_1111;
            3'b011: led = 8'b1110_1111;
            3'b100: led = 8'b1111_0111;
            3'b101: led = 8'b1111_1011;
            3'b110: led = 8'b1111_1101;
            3'b111: led = 8'b1111_1110;
        endcase
    end
endmodule

```

功能扩展

- 4 线至 16 线译码器（串行拓展）

  片　的 S_B 与片　的 S_A 并接在一起作为最高位 D 的输入端。片　的 S_C 与片　的 S_B 连在一起作为使能端。

- 6 线至 64 线译码器（并行拓展）

  在 8 片 74LS138 的基础上添加 1 片 74LS138 作为高 3 位译码器，将其输出结果输入到 8 片 74LS138 的 S_B、S_C 端口。

- 地址译码器

  将地址低 3 位输入到 $A_2 \sim A_0$ 端，其他位输入到 S_A、S_B、S_C 端口，保证始终正常工作。

  可以将所有 1 通过与门，所有 0 通过或门，接入使能端，即可实现地址译码器。

- 数据分配器

  将数据 D 输入到 S_B、S_C 端口。

  当 $D=1$ 时，译码器所有端口输出 1，数据分配器分配的数据必定是 1；当 $D=0$ 时，译码器对应端口输出 0。

- 函数发生器

  可以将译码器的输出端看作 2^n 个最小项，可以通过基本逻辑门组合实现任意逻辑函数。

  注意：译码器为**负逻辑输出**！

【技巧】最小项间转换

$m_0 + m_1 + m_2 = \overline{m_3}$

运用此式可以化简逻辑函数，减少逻辑门数量。

3. 二——十进制译码器（4 线-10 线译码器）

   功能扩展

   - 3 线至 8 线译码器

     f_8、f_9 不用即可。

4. 七段荧光数码管显示器 74LS48

   共阴极：统一接低电平，输出端口为正逻辑，即 1 亮 0 灭；

   共阳极：统一接高电平，输出端口为负逻辑，即 0 亮 1 灭；

   译码器将 8421BCD 码映射为七段荧光数码管的七个段的点亮情况。

   除去 4 输入 7 输出端口外，有如下特殊端口：

   - $\overline{BI}$：强制灭灯，当 $\overline{BI} = 1$ 时，强制全灭，优先级最高；
   - $\overline{LT}$：试灯，当 $\overline{LT} = 1$ 时，强制全亮；
   - $\overline{RBI}$：灭 0 输入信号，当 $\overline{RBI} = 1$ 时灭 0；
   - $\overline{RBO}$：灭 0 输入信号，当本数码管显示 0 时输出 0；

   只需在整数部分把高位的 $\overline{RBO}$ 与低位的 $\overline{RBI}$ 相连，在小数部分将低位的 $\overline{RBO}$ 与高位的 $\overline{RBI}$ 相连，就可以熄灭前后多余的 0。

（四）数据选择器

1. TTL 8 选 1 数据选择器 74LS151

   地址端 A_2, A_1, A_0，数据端 $D_7 \sim D_0$，使能端 $\overline{ST}$ 输出端 Y 和 $\overline{W}$。

2. 双 4 选 1 数据选择器 74LS153

   地址端 A_1 和 A_0，数据端 $D_3 \sim D_0$，使能端 $\overline{1ST}$ 和 $\overline{2ST}$，输出端 $1Y$ 和 $2Y$。

```
1  module mux4_to_1(out, i0, i1, i2, i3, s0, s1);
2
3      output out;
4      input i0, i1, i2, i3, s0, s1;
5      reg out;
6
7      always @(s1 or s0 or i0 or i1 or i2 or i3)
8                                  // 用always块建立组合逻辑模型时，要用阻塞赋值
9      begin
10         case({s1, s0})
11             2'b00: out = i0;
12             2'b01: out = i1;
```

```
            2'b10: out = i2;
            2'b11: out = i3;
            default: out = 1'bx; // 其他情况下输出1位任意值
        endcase
    end

endmodule

```

功能扩展

- 8 选 1 数据选择器

  将 A_2 与其反变量输入 $\overline{1ST}$ 与 $\overline{2ST}$，对输出求或。

- 16 选 1 数据选择器

  将 A_0 和 A_1 输入第一级电路，16 中选 4，然后再将结果输入 2 级电路，用 A_2 和 A_3 进行选择，从 4 个输出中选 1。

- 并串转换电路

  在地址端输入 $00 \to 01 \to 10 \to 11 \to 00 \to \cdots$，即可输出 $D_0D_1D_2D_3\cdots$。

- 函数发生器

  若 $m \le n$，则将 2^m 个最小项输入到地址端。

  若 $m > n$，则将 2^n 个最小项输入到地址端，剩余的最小项通过逻辑门组合后输入到数据端。一般选择 2^n 个最小项中最多的作为数据端输入，剩余的作为地址端输入。

> **Karnaugh 图法**
>
> 根据地址端的最小项圈出 Karnaugh 图中的子 Karnaugh 图，再在子 Karnaugh 图中圈出最小项，最后将圈出的最小项通过逻辑门组合后输入到数据端。

（五）数据分配器

1. TTL 双 1 线至 4 线数据分配器 74LS155

   可以视为 2 到 4 线译码器添加了使能端与数据输入端的 $\overline{D}$ 与门。

   注意：上下使能端的**正负逻辑相反**。

```
module demux4(y0, y1, y2, y3, din, a);

    input din;
    input [1:0] a;
    output reg y0, y1, y2, y3;

    always @(din, a)
    begin
        y0 = 0; y1 = 0; y2 = 0; y3 = 0;
                                // 用always块建立组合逻辑模型时，要用阻塞赋值
```

```
        case(a)
            2'b00: y0 = din;
            2'b01: y1 = din;
            2'b10: y2 = din;
            2'b11: y3 = din;
        endcase
    end

endmodule

```

功能扩展

- 8 选 1 数据分配器

  将 A_2 与其反变量输入 $\overline{1ST}$ 与 $\overline{2ST}$，对输出求或。

- 译码器

  令 $\overline{D}=0$，即可实现译码器功能。

第四章　Verilog 硬件描述语言（Verilog HDL）

Verilog HDL 既是一种行为描述语言也是一种结构描述语言，即既可以**用电路的功能描述**，也可以**用元器件及其之间的连接**来建立 Verilog HDL 模型。

Verilog HDL 可以描述系统级、算法级、寄存器传输级（RTL）、门级和开关级五种级别的电路。其中前三种称为行为描述，后两种称为结构描述。

一、数据类型

（一）数值

1. 数字

   表示：位数 +'+ 数制 + 数值。例：`4'b1010` 表示 4 位二进制数 1010，`8'hFF` 表示 8 位十六进制数 FF。

   数制：`b` 表示二进制，`o` 表示八进制，`d` 表示十进制，`h` 表示十六进制。

   特殊：

   - 负号要放在最前面，如`-4'b1010`;
   - 不写位数默认为最长值，不写数制默认为十进制;
   - `'1` 表示全 1，`'0` 表示全 0。

2. 字符串

   可以看作特殊的长度为 8 bit 的二进制数，`"A"` 表示 8 位二进制数 01000001。字符串的长度为 8 × 字符个数bit。

3. 参数 `parameter`

（二）变量

`wire` 常用于 `assign` **语句**的左值；

`reg` 常用于 `always` **块中** `=` **或** `<=` **语句**的左值。

此外还有 `memory` 等类型，不多介绍。

二、运算符

大部分运算符与 C 语言相同。下面介绍几种不一样的：

1. 拼接运算符 `{}`

   直接把所有表达式的值按序拼接。

2. 阻塞赋值和非阻塞赋值

   阻塞赋值的符号是 `=`，**只用于组合逻辑电路中**，即 `assign` 语句或 `always @ *`；

   非阻塞赋值的符号是 `<=`，**只用于时序逻辑电路中**，即 `always @ (posedge clk)` 或类似语句。

三、语句

1. 赋值

   - 连续赋值 `assign`

     用于组合逻辑电路的连续赋值，**左值必须是** `wire`。

   - `always` 块

     用于组合逻辑电路或时序逻辑电路的过程赋值，**左值必须是** `reg`。

   - `initial` 块

     用于仿真初始化，**左值必须是** `reg`。

2. 条件

   `0`、`x`、`z` 均为逻辑 0，`1` 为逻辑 1。

3. 循环

   除 `while`、`for` 外，还有 `repeat`、`forever`。

   - `repeat` 语句：重复执行指定次数的循环体；
   - `forever` 语句：重复执行循环体，直到仿真结束。

第五章　Karnaugh 图与其数学结构